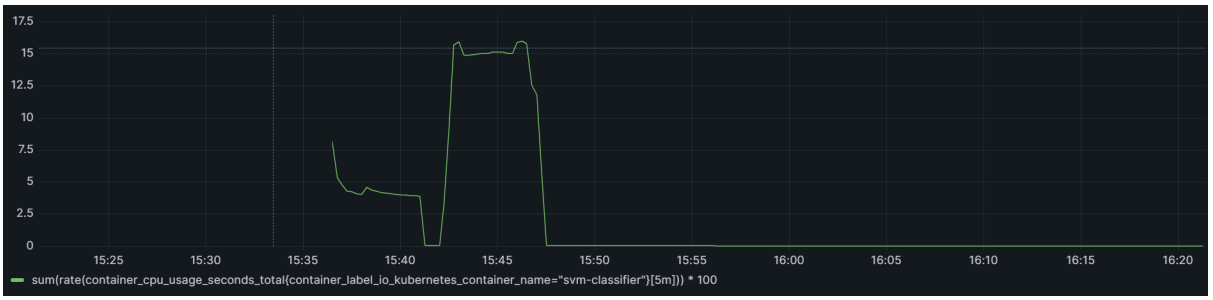
For 2000 samples

The weird gaps and different peaks in graphs are results of running things several times for debugging purposes

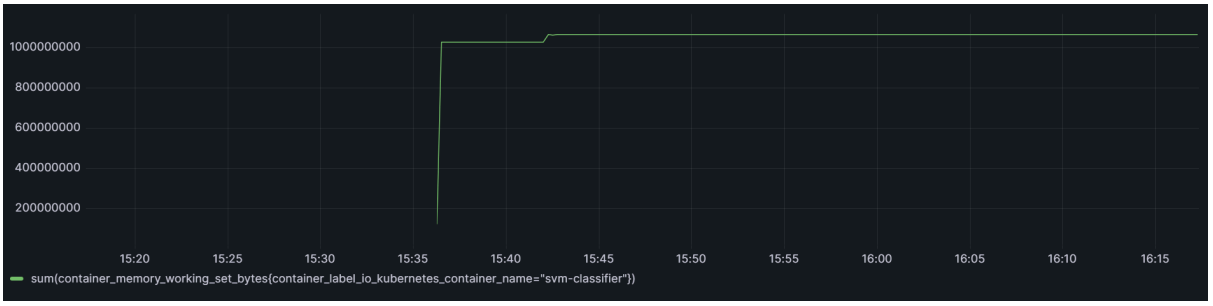
Best Parameters:

```
C:\Users\Charm\Downloads\LabTest2-Code>curl -X POST http://localhost:5002/api/classify
{"accuracy":0.8975,"best_params":{"svm__C":10,"svm__gamma":0.001}}
```

CPU Usage in percentage:



Memory Usage in MiB:



Tuning Latency in ms:

Training Latency in ms:

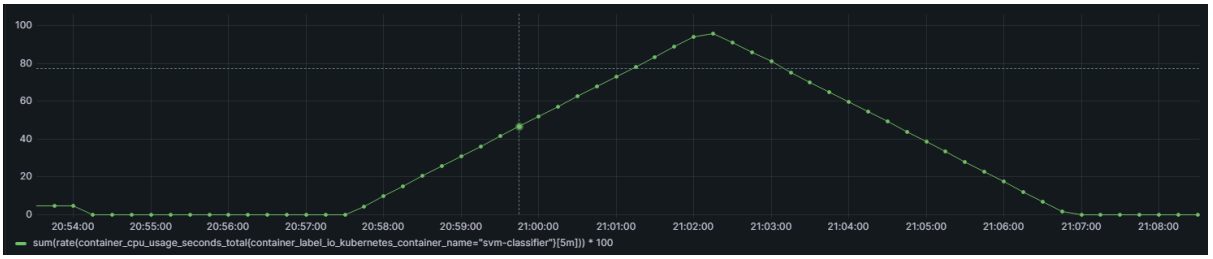
Evaluation Latency in ms:

For 8000 samples

Best Parameters:

```
C:\Users\Charm\Downloads\LabTest2-Code>curl -X POST http://localhost:5002/api/classify
{"accuracy":0.944375,"best_params":{"svm__C":10,"svm__gamma":0.001}}
```

CPU Usage in Percentage:



Memory Usage in MiB:

Tuning Latency in ms

Training Latency in ms:

Evaluation Latency in ms:

Comment on the two sets of results. What can you tell about the performance of the service and its scalability. What is the bottleneck of the service? (10 points)

- The CPU usage of the SVM service remains moderate but spikes significantly during training and tuning stages especially for 8000 samples.
- Memory usage increases around the same stages and is doubled for 8000 samples.
- Between requests, CPU usage drops close to idle, and memory stabilizes, indicating that the service does not maintain heavy background processes.
- The service performs well for small batches, with moderate latency and manageable CPU/memory usage.
- As the dataset grows, tuning latency becomes the main bottleneck, dominating response time and causing CPU spikes, even for low sample counts due to retraining the classifier upon each request.
- The SVM model itself is CPU-bound during tuning and training, making it difficult to scale horizontally unless requests are handled asynchronously or hyperparameter search is optimized.

Structure and Solution Explanation

Dockerfile

```
FROM python:slim

WORKDIR /app

RUN pip install flask scikit-learn prometheus_client

COPY ./svm/requirements.txt ./svm/requirements.txt
RUN pip install -r ./svm/requirements.txt

COPY ./svm ./svm

EXPOSE 5002

CMD ["python", "./svm/svm_classifier.py"]
```

- Slim python image for less space
- Installs dependencies
- Exposes port 5002
- Executes code

Deployment (svm-classifier-deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: svm-classifier-deployment
  labels:
    app: svm-classifier
spec:
  replicas: 1
  selector:
    matchLabels:
      app: svm-classifier
  template:
    metadata:
      labels:
        app: svm-classifier
    spec:
      containers:
      - name: svm-classifier
        image: toasty106/svm_classifier:latest
        ports:
        - containerPort: 5002
```

- Acts as a blueprint for the SVM service. It ensures high availability (via replicas), self-healing, and links to the network via containerPort: 5002.
- Must be pushed to a registry (e.g., `toasty106/svm_classifier:latest`) for K8s to pull it.
- Manages svm-classifier pods

Services

```yaml
apiVersion: v1
kind: Service
metadata:
  name: svm-classifier-service
spec:
  selector:
    app: svm-classifier
  ports:
    - protocol: TCP
      port: 5002
      targetPort: 5002
```

- Provides a consistent DNS name for your application, which allows for communication despite ephemeral nature of individual pods
- Acts as a gateway that forwards incoming traffic from a specified service port to the application's container port
- Uses label selectors to dynamically discover and direct incoming requests to healthy, matching pods.
- Automatically distributes incoming requests across all available replicas, ensuring high availability and balanced workloads.

svm_classifier.py

```python
tuning_metric = Histogram('tuning_latency_seconds', "Tuning latency")
training_metric = Histogram('training_latency_seconds', "Training
latency")
evaluate_metric = Histogram('evaluate_latency_seconds', "Evaluate
latency")
```

- Initializes a measurement tool which will track how long specific parts of code take to execute
- We use histograms as it records average time and distribution of times
- The first argument is the metric name used in the PromQL code for graphing through Grafana

```
app = Flask(__name__)

@app.route("/metrics", methods=["GET"])
def metrics():
    return generate_latest(), 200, {'Content-Type':
CONTENT_TYPE_LATEST}
```

- sets up the web interface for prometheus
- Allows for GET requests to be sent to /metrics to scrape data
- Prometheus reads data and displays on web interface

```
@app.route("/api/classify", methods=["POST"])
def api_classify():
```

- Application endpoint
- Sends POST request and triggers api_classify function
- Returns the model's accuracy and parameters

Prometheus Configuration (prometheus-config.yaml)

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: prometheus-config
data:
  prometheus.yml: |
    global:
      scrape_interval: 15s
    scrape_configs:
      - job_name: 'prometheus'
        static_configs:
          - targets: ['localhost:9090']
      - job_name: 'svm-classifier'
        static_configs:
          - targets: ['svm-classifier-service:5002']
      - job_name: 'kubernetes-cadvisor'
        static_configs:
          - targets: ['cadvisor:8080']
```

- Tells prometheus to scrape and refresh data every 15 seconds
- Monitors itself
- Directs it to look for svm-classifier-service at port 5002
- Also scrapes cAdvisor for CPU and memory usage statistics
- Acts as a bridge and helps with decoupling as you keep the monitoring configuration separate from the Prometheus deployment itself